Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 2do. Semestre 2026

SISTEMA DE AUTORIZACIÓN DE DOCUMENTOS TRIBUTARIOS ELECTRÓNICOS

202201476 – María Andrea Charuc López

Resumen

El ensayo describe el diseño y la construcción de un sistema que simula el servicio de la Superintendencia de Administración Tributaria (SAT) para autorizar Documentos Tributarios Electrónicos (DTE), un tema vigente en Guatemala por la obligación de emitir factura electrónica. La solución se divide en dos programas: una API REST desarrollada en ASP.NET Core con .NET 10, que valida contribuyentes y facturas, genera códigos de aprobación y calcula reportes, y una aplicación web en Razor Pages que consume la API y presenta los resultados con tablas, gráficas y reportes en PDF. Se exponen las decisiones de diseño orientado a objetos, los Tipos de Dato Abstracto propios (lista enlazada, tabla hash y lista ordenada), los algoritmos principales (validación del NIT con módulo 11, extracción de fechas con expresiones regulares y correlativo diario) y la persistencia en archivos JSON. Se concluye que separar la lógica de negocio en un servicio independiente y construir estructuras propias facilita probar, mantener y extender el sistema.

Palabras clave

API REST, Razor Pages, POO, TDA, módulo 11.

Abstract

This essay describes the design and construction of a system that simulates the service used by the Guatemalan tax authority (SAT) to authorize Electronic Tax Documents (DTE), a current topic in Guatemala because electronic invoicing is mandatory. The solution is split into two programs: a REST API built with ASP.NET Core on .NET 10, which validates taxpayers and invoices, generates approval codes and computes reports, and a Razor Pages web application that consumes the API and displays the results with tables, charts and PDF reports. The essay explains the object-oriented design decisions, the custom Abstract Data Types (linked list, hash table and sorted list), the main algorithms (tax ID validation with modulo 11, date extraction with regular expressions and a daily sequence number) and the persistence in JSON files. It concludes that keeping the business logic in an independent service and building custom data structures makes the system easier to test, maintain and extend.

Keywords

REST API, Razor Pages, OOP, ADT, modulo 11.

Introducción

La facturación electrónica exige que cada factura sea revisada y autorizada por la administración tributaria antes de tener validez. Para ello se necesita un sistema que reciba solicitudes, verifique que los contribuyentes existan, que los montos estén bien calculados y que no se repitan documentos, y que devuelva un código único de aprobación.

El propósito de este ensayo es mostrar cómo se construyó una versión simplificada de ese sistema aplicando programación orientada a objetos en C#, algoritmos propios y el modelo cliente-servidor. ¿Cómo se valida un NIT sin consultar a la SAT? ¿Qué estructura conviene para buscar contribuyentes rápidamente? Las siguientes secciones responden estas preguntas al presentar la arquitectura, el modelo de clases, los algoritmos, la persistencia y la interfaz de usuario.

Desarrollo del tema

a. Arquitectura de la solución

La solución sigue una arquitectura de dos programas que se comunican por HTTP. El Backend es una API web de ASP.NET Core que expone servicios REST y responde en formato JSON. El Frontend es una aplicación de Razor Pages que funciona como simulador del cliente: el usuario interactúa con ella desde el navegador y ella consume la API por medio de HttpClient. Separar ambos programas deja la lógica de negocio en un solo lugar y permite que cualquier otro cliente, como Postman, use los mismos servicios (Figura 1).

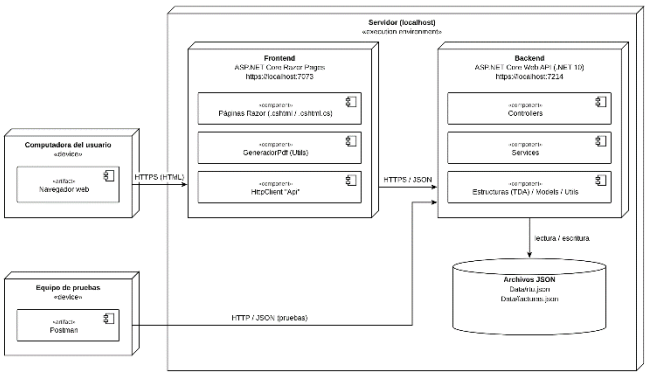


Figura 1. Diagrama de despliegue de la solución.
Fuente: elaboración propia.

Dentro del Backend el código se organiza en cinco carpetas: Models contiene las clases que representan los datos; Estructuras contiene los TDA propios; Utils contiene las clases con los algoritmos de validación; Services contiene la lógica de negocio y el almacenamiento; y Controllers recibe las peticiones HTTP y delega el trabajo a los servicios. Los servicios se registran como Singleton en el contenedor de inyección de dependencias, de modo que todas las peticiones comparten la misma información en memoria. La Tabla I resume los servicios que ofrece la API.

Los tres servicios POST reciben o modifican datos, mientras que los servicios GET solo consultan la información almacenada. Todas las respuestas usan los nombres de campos que define el enunciado.

b. Modelo orientado a objetos

Cada concepto del problema se representó con una clase. Contribuyente guarda el NIT y el nombre; SolicitudFactura refleja exactamente el mensaje de entrada; y Factura representa una solicitud ya procesada, con su fecha, montos, código de aprobación y la lista de errores encontrados. La propiedad calculada Aprobada devuelve verdadero

solo cuando la lista de errores está vacía, lo que evita guardar el mismo dato dos veces.

Las respuestas también son clases propias (RespuestaRtu, RespuestaAutorizacion, ReporteFecha, ConsultaContribuyente, ResumenIva y ResumenRango), y el atributo JsonPropertyName garantiza que los nombres del JSON coincidan con los del enunciado. RtuService encapsula el registro de contribuyentes, FacturaService depende de RtuService para validar los NIT y ReporteService depende de FacturaService para calcular los reportes. El diagrama de clases completo se presenta en el Apéndice A.

c. Tipos de Dato Abstracto propios

Se implementaron tres TDA propios en la carpeta Estructuras. Ninguno usa internamente las colecciones de .NET: se construyen con nodos enlazados y arreglos, y exponen solo las operaciones que necesita el programa.

ListaEnlazada<T> es una lista simple que mantiene referencias al primer y al último nodo, por lo que agregar al final no requiere recorrerla. Ofrece operaciones para agregar, buscar, eliminar y copiar, e implementa ICollection<T> para recorrerse con foreach y para que el serializador JSON la lea y escriba como un arreglo. Guarda las facturas en orden de llegada, los errores de cada factura y los listados de las respuestas.

TablaHash<TValor> usa como llave un texto. La posición se calcula con una función propia que recorre los caracteres aplicando hash = hash · 31 + carácter y reduce el resultado con la operación módulo. Las colisiones se resuelven por encadenamiento y, al llegar al 75 % de ocupación, el arreglo se duplica. Guarda el RTU, donde la operación más frecuente es preguntar si un NIT existe, y cuenta emisores y receptores distintos (Apéndice B.4).

ListaOrdenada<TClave, TValor> mantiene sus nodos ordenados por llave. Su operación ObtenerOCrear devuelve el valor de una llave existente o inserta uno nuevo en la posición correcta, de modo que los reportes agrupados por fecha o por NIT salen ordenados sin un algoritmo de ordenamiento adicional.

d. Validación del NIT con módulo 11

Primero, una expresión regular verifica que el NIT tenga solo dígitos seguidos de un verificador que puede ser un número o la letra K. Luego se recorren los dígitos de derecha a izquierda, multiplicando cada uno por su posición a partir de dos, y se calcula el verificador esperado con la ecuación (1):

$$v = (11 - (\sum_{i=1}^{n} d_i \cdot p_i) \bmod 11) \bmod 11 \quad (1)$$

donde:

v = dígito verificador esperado (si v = 10 se usa K)

d*i* = dígito en la posición i, contando desde la derecha

p*i* = peso de la posición, p*i* = i + 1

n = cantidad de dígitos sin el verificador

Por ejemplo, para el NIT 7378106 la suma es 6·2 + 0·3 + 1·4 + 8·5 + 7·6 + 3·7 + 7·8 = 175; como 175 mod 11 = 10, el verificador es (11 − 10) mod 11 = 1, igual al último dígito, por lo que el NIT es válido. El diagrama de actividades se muestra en el Apéndice B.1.

e. Extracción de la fecha con expresiones regulares

El campo TIEMPO puede incluir texto adicional, como el nombre de un lugar. Para no tratarlo como error se usa la expresión regular, que localiza una fecha con el formato exacto dd/mm/aaaa hh:mi. Las

búsquedas negativas al inicio y al final evitan tomar parte de un número más largo. Después se valida que la hora esté entre 0 y 23, los minutos entre 0 y 59 y que la fecha exista; si algo falla, la factura se marca con FECHA_INVALIDA.

f. Autorización de facturas y código de aprobación

Cada solicitud pasa por validaciones independientes y cada problema se agrega a la lista de errores de la factura: fecha inválida, referencia vacía o ya usada por cualquier factura anterior, NIT emisor o receptor inválido o inexistente en el RTU, valor no positivo o con más de dos decimales, e IVA o TOTAL mal calculados según la ecuación (2):

$$IVA = \text{round}(0.12 \cdot VALOR, 2),\ TOTAL = VALOR + IVA \quad (2)$$

El TOTAL se compara contra la suma del VALOR y el IVA recibidos, de modo que cada error se cuenta por separado. Una factura con cualquier error se rechaza, pero todas se guardan para los reportes.

Cuando una factura se aprueba se genera un código con el formato yyyymmdd########. El algoritmo usa la fecha de procesamiento como prefijo, busca el mayor correlativo con ese prefijo y le suma uno, así el correlativo reinicia cada día sin un contador adicional (Apéndices B.2 y B.3).

g. Consultas, reportes y persistencia

ReporteService calcula cuatro reportes. El reporte por fechas cuenta, por día, las facturas recibidas, cada tipo de error, las facturas sin errores y la cantidad de emisores y receptores distintos. La consulta de contribuyente acumula por día la cantidad y el monto de las facturas emitidas o recibidas por un NIT; el resumen de IVA muestra el IVA emitido y recibido por NIT en una fecha; y el resumen por rango suma por día el valor total o sin IVA. Para no perder la información, RtuService y FacturaService guardan sus datos en Data/rtu.json y Data/facturas.json con System.Text.Json, y los cargan al iniciar el servicio.

h. Interfaz de usuario en Razor Pages

La página de inicio muestra un botón por cada opción: Enviar mensaje, Consultar contribuyente, Resumen IVA, Resumen por rango, Reporte por fechas e Inicializar; el menú superior conserva Ayuda, que muestra los datos de la estudiante y la documentación del programa. Enviar mensaje permite elegir un archivo .json y muestra la respuesta del Backend. Las páginas de consulta presentan filtros, tablas y un botón para descargar el reporte en PDF.

Las gráficas de los dos resúmenes se construyeron sin librerías externas: el modelo de página calcula el porcentaje de cada valor respecto del máximo y la vista dibuja barras con ese ancho. De igual forma, la clase GeneradorPdf escribe el archivo PDF según la especificación del formato: crea el catálogo, las páginas, las fuentes y el contenido, agrega páginas cuando el texto no cabe y construye la tabla de referencias cruzadas.

i. Pruebas realizadas

Cada servicio se probó con el archivo Backend.http y con Postman, y después desde la interfaz web. Con un archivo de once contribuyentes se obtuvieron siete nuevos, uno actualizado y tres NIT inválidos; con un archivo de doce facturas, las que tenían NIT inexistente, IVA mal calculado o referencia repetida fueron rechazadas con el código de error correcto. Finalmente se verificó que las consultas, las gráficas y los PDF mostraran los mismos totales calculados a mano.

Conclusiones

- Separar la solución en una API y un cliente web concentró las reglas de negocio en el Backend y permitió probarlas de forma independiente, incluso desde Postman.
- El diseño orientado a objetos, con modelos, servicios y controladores de responsabilidades claras, facilitó agregar nuevas consultas sin modificar el código ya probado.
- Implementar TDA propios permitió elegir para cada problema la estructura adecuada: la tabla hash para búsquedas por NIT y la lista ordenada para reportes cronológicos.
- Las expresiones regulares y el algoritmo de módulo 11 resolvieron la validación de datos de texto libre sin depender de servicios externos.

Queda abierta la pregunta de cómo escalaría el sistema con millones de facturas diarias; una base de datos y procesamiento en paralelo serían el siguiente paso natural.

Referencias bibliográficas

Adobe Systems. (2008). *Document management — Portable document format — Part 1: PDF 1.7* (ISO 32000-1).

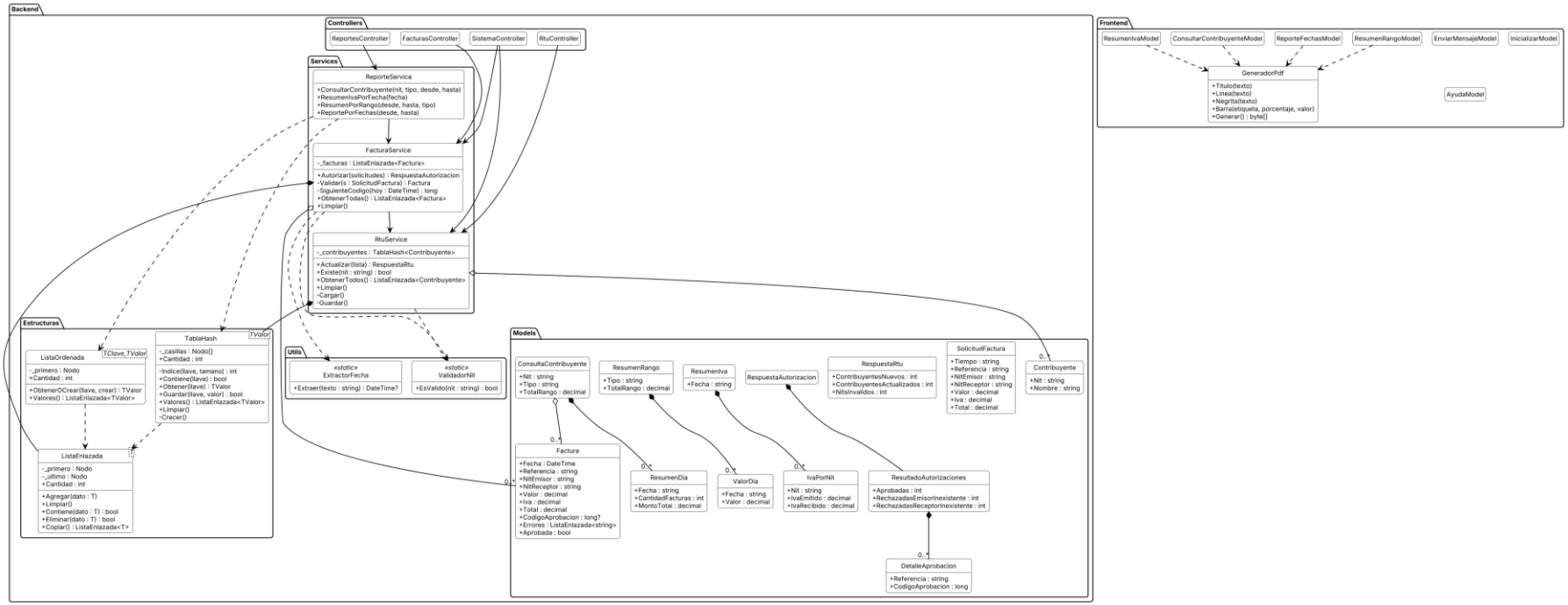
Cormen, T. H., Leiserson, C. E., Rivest, R. L. y Stein, C. (2009). *Introduction to Algorithms* (3.ª ed.). MIT Press.

Microsoft. (2025). *Documentación de ASP.NET Core y Razor Pages*. https://learn.microsoft.com/es-es/aspnet/core/

Superintendencia de Administración Tributaria. (2026). *Factura Electrónica en Línea (FEL)*. https://portal.sat.gob.gt/

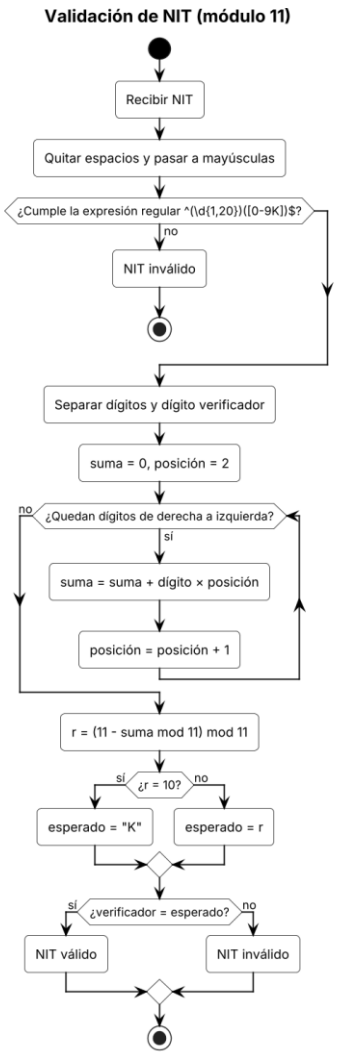
Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 2do. Semestre 2026

Apéndice A. Diagrama de clases

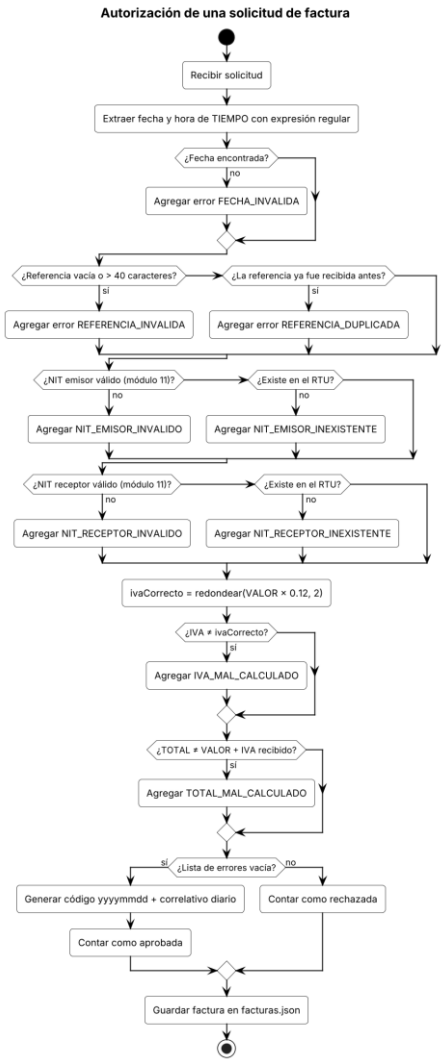


Fuente: elaboración propia.

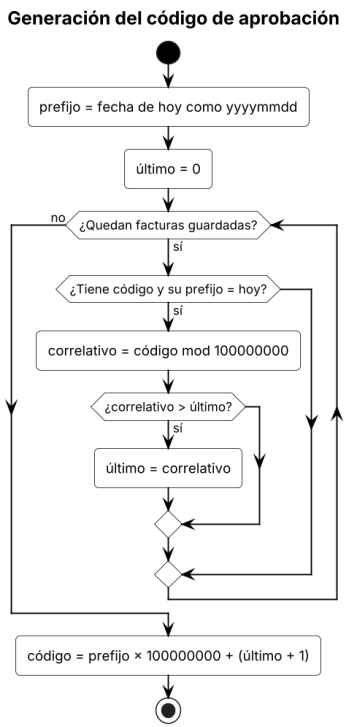
Apéndice B. Diagramas de actividades de los principales algoritmos



B.1 Validación del NIT



B.2 Autorización de una factura



B.3 Código de aprobación

Fuente: elaboración propia.